

Project Documentation

Ludo Advanced

To the course 'Projektpraktikum'
in the study program 'Computer Science'

submitted to the Department of
Computer Science and Business Informatics
of the
Provadis School of International Management and Technology
from

Name: Lars Bolender, Fabian Gemming, Benedict Glatz

Matriculation number: E053, E063, E151

Study program: BIN24

E-Mail: lars.bolender@stud-provadis-hochschule.de
fabian.gemming@stud-provadis-hochschule.de
benedict.glatz@stud-provadis-hochschule.de

Reviewer: Prof. Dr. Lamya Abdullah

End of the processing period: 15.09.2026

Table of Contents

List of Tables	v
List of Figures.....	vi
1 Executive Summary	1
1.1 Project Overview	1
1.2 Main Objectives	1
1.3 Key Outcomes.....	1
1.4 Technologies Used.....	1
2 Introduction.....	2
2.1 Background and Motivation	2
2.2 Project Objectives	2
2.3 Scope and Limitations	3
2.4 Target Users / Stakeholders	3
3 Requirement Analysis	4
3.1 Use Cases / User Stories	4
3.2 Functional Requirements	5
3.3 Non-Functional Requirements	6
4 Project Management and Team Organization.....	8
4.1 Team Structure and Roles	8
4.2 Development Process Model	8
4.3 Schedule and Timeline.....	9
4.4 Collaboration and Communication	9
4.5 Risk Management	10
4.6 Progress Tracking	10
5 System Analysis and Design.....	10
5.1 Domain Analysis.....	10
5.2 System Overview	11
5.3 Design Decisions	11
5.3.1 Relative Position Coordinate Topology (r)	11
5.3.2 Risk-Reward Dice Pool Dynamics (D2 to D20)	12
5.3.3 Synchronous Reaction Window Architecture	12
5.4 UI/UX Design	12
5.5 Prototype Evolution	14
6 Software Architecture	14
6.1 Architectural Overview.....	14

6.2 Architectural Decisions.....	15
6.3 Component Diagram.....	16
6.4 Frontend Design.....	16
6.5 Backend Design	17
6.6 Runtime / Deployment View	17
7 Data Design.....	18
7.1 Data Model	18
7.2 Entity Relationship Diagram.....	18
7.3 Data Sources and Data Flow.....	18
8 Implementation	18
8.1 Development Environment	18
8.2 Project Structure	18
8.3 Key Features	18
8.4 Selected Implementation Details	18
8.5 Use of External Libraries and AI Tools.....	18
9 Testing and Quality Assurance.....	19
9.1 Testing Strategy	19
9.2 Test Cases	19
9.3 Test Results.....	20
9.4 Defects and Corrections.....	21
9.5 Quality Improvements	21
10 Reflection and Evaluation.....	21
10.1 Goal Achievement	21
10.2 Technical Lessons Learned	22
10.2.1 Headless Architecture enables Rapid Test Automation	22
10.2.2 Normalized Relative Coordinate Spaces Reduce Cyclomatic Complexity....	22
10.2.3 Empirical Verification of Tooling Assumptions prevents False Defect Reports	
Error! Bookmark not defined.	
10.3 Project Management Lessons Learned	22
10.4 Teamwork Reflection	23
10.5 Future Improvements.....	Error! Bookmark not defined.
11 Conclusion	23
11.1 Summary.....	23
11.2 Open Issues.....	23
11.3 Future Work.....	24
12 Appendix.....	25
12.1 Tables	25

12.2	Figures	28
13	AI Directory	31

List of Tables

Table 1: Stakeholder Overview	3
Table 2: User Stories.....	4
Table 3: Non-Functional Requirements.....	7
Table 4: Team Structure and Roles.....	8
Table 5: Risks and Mitigation Strategies	10
Table 6: Coverage Results	20
Table 7: Objectives and Evaluation	22
Table 8: Key Outcomes	25
Table 9: SMART-analysis	26
Table 10: Main Requirements and MoSCoW-Analysis	27

List of Figures

Figure 1: Traditional Ludo vs. Ludo Advanced	2
Figure 2: Use Case Diagram	5
Figure 3: Backlog Epics.....	6
Figure 4: Schedule and Timeline	9
Figure 5: State Machine	11
Figure 6: Prototype Evolution Overview	14
Figure 7: Architecture Layers	15
Figure 8: Component Diagram	16
Figure 9: Deployment View	Error! Bookmark not defined.
Figure 10: Testing Pyramid	19
Figure 11: Testing, Count and Pass Rate	20
Figure 12: Kanban-Board Overview.....	28
Figure 13: Gantt-Diagram Overview	28

1 Executive Summary

1.1 Project Overview

Ludo Advanced is a 2D web-based strategy game for 1 - 4 players that revamps **Mensch ärgere Dich nicht** through tactical card systems. Players draft from a **Dice Card Pool** (D2 to D20) to change roll probabilities and deploy offensive Actions or defensive Reactions via a **Skill Card Pool**. These mechanisms layer over the classic 40-tile circuit, where pawns still require maximum die rolls to leave the yard, capture opponent pieces, and race into dedicated four-slot home columns.

1.2 Main Objectives

The project shifts classic roll-and-move luck into balanced strategical decision-making. Developed under an agile **Scrum** framework, the project prioritized complete traceability and automated testing targeting at least 80% coverage.

1.3 Key Outcomes

The final delivery achieved a local hot-seat **Minimum Viable Product** (MVP) with full English and German localization. **Quality targets** were substantially exceeded, reaching 99.19% line coverage across the core rule engine and state machine (742/748 lines) via 503 unit tests across 34 suites, alongside 42 cross-browser Playwright end-to-end flows. **Code modularity** remained strictly within the 300-line-per-file ceiling across the entire repository. For a detailed overview, see Table 8: Key Outcomes. #TODO Update Numbers

1.4 Technologies Used

The client-side application uses standard **JavaScript** (ES Modules) with a lightweight frontend framework, utilizing **jQuery** 4.0 for DOM rendering and event delegation, alongside **il18next** for localization. The build and toolchain rely on **Vite 8.2**, **ESLint**, and **Prettier**. Testing is powered by **Vitest 4.1** with the v8 coverage provider for headless verification and **Playwright** 1.62 for browser automation, managed via **GitHub Projects** v2 and **Conventional Commits**. Anthropic **Claude** Opus 5 and Fable 5.1 were used for AI-assisted planning and implementation. Communication took place via Microsoft **Teams**. Google **Gemini** 3.8 Flash and Microsoft **Word** were used for creating the documentation, while Microsoft **Powerpoint** was used for creating the presentation.

2 Introduction

2.1 Background and Motivation

Ludo, widely known in German-speaking regions as **Mensch ärgere Dich nicht**, has been a popular staple of family board gaming for over a century. Although the traditional game is very successful, its excessive reliance on pure chance can diminish player satisfaction. The player's entire repertoire is a standard six-sided die. This dependency often results in frustrating starting stalls, where a player unable to roll a six remains trapped in the starting yard for an extended period without being able to participate in the match. Furthermore, even when pieces are active on the board, the decision space remains severely limited, as players frequently have only a single legal pawn to advance, reducing strategical play to near zero.

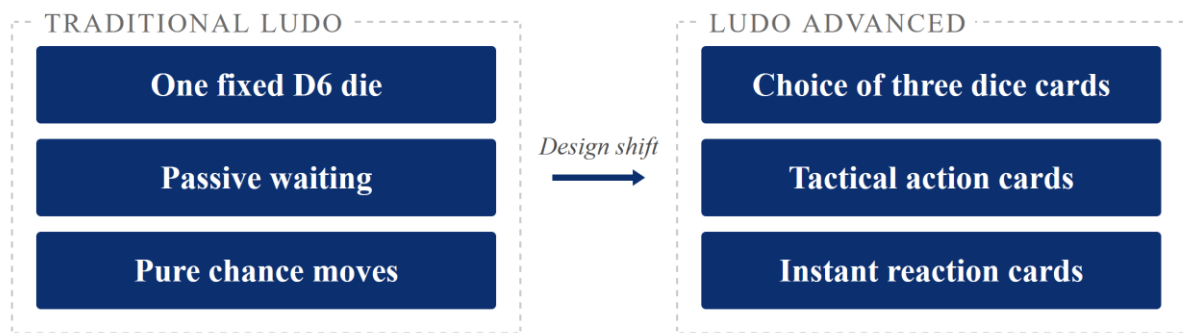


Figure 1: Traditional Ludo vs. Ludo Advanced

Ludo Advanced addresses these fundamental design flaws directly by blending traditional board topology with modern deck-building and hand-management concepts. The fixed die is replaced with a dynamic **Dice Card Pool** containing dice ranging from D2 to D20, paired with a specialized **Skill Card Pool** containing Action and Reaction cards. This restructuring transforms passive waiting into active risk calculation and strategical maneuvering. Players continuously evaluate probability trade-offs, manipulate movement distances, and actively counter opponent attacks, **revitalizing a timeless classic**.

2.2 Project Objectives

The project was executed as part of the **Projektpraktikum** module within the Computer Science curriculum at the Provadis School of International Management and Technology during the summer semester of 2026. The technical and organizational goals of the project were defined according to the **SMART** criteria framework (See Table 9: SMART-analysis).

The primary product goal was the delivery of a fully playable **Minimum Viable Product** supporting one to four local hot-seat players. **Architectural requirements** included a strictly decoupled three-layer structure consisting of game rules in *src/core/*, an isolated state management machine in *src/state/*, and a dynamic DOM rendering layer in *src/ui*.

To guarantee maintainability and high software reliability, the team established an automated quality objective requiring at least **80% line test coverage** across all core rule and state modules. Finally, project management objectives emphasized strict agile execution, featuring weekly reviews, per-commit documentation updates, and complete prompt logging for all generative artificial intelligence interactions.

2.3 Scope and Limitations

To deliver a polished and reliable software artifact within an eight-week university project timeframe, functional boundaries were established using **MoSCoW** prioritization. The specific decisions are concretized in chapter 3.2 Functional Requirements.

2.4 Target Users / Stakeholders

The development was guided by academic and practical stakeholders, each contributing specific expectations and constraints to the project lifecycle:

Stakeholder Group	Role and Primary Interest	Impact on Project Execution
Module Reviewer (Prof. Dr. Lamya Abdullah)	Academic supervision, evaluation of software engineering practices, architecture, and project management guidelines.	High influence: grading guidelines established documentation depth, test coverage, and architectural requirements.
Development Team (Lars Bolender, Fabian Gemming, Benedict Glatz)	Product Owner and Scrum developers responsible for engineering, testing, and documenting the system.	High influence: daily collaborative decision making, code authoring, peer reviews, and sprint execution.
Peer Students & Playtesters	User experience testing, game balance verification, rule clarity, and fun factor assessment.	Medium influence: qualitative feedback on UI responsiveness and card effects.
Casual Board Game Players	End users seeking engaging, accessible, and fast-paced web board games.	Guided design: zero-installation browser access, intuitive visual move hints, and clear rules.

Table 1: Stakeholder Overview

3 Requirement Analysis

3.1 Use Cases / User Stories

The requirements engineering process focused on modeling the complete user journey of a local match, capturing interactions from initial match configuration to final victory conditions. Use cases were formulated as agile user stories to define behavioral expectations and acceptance criteria. These are then linked to functional requirements:

ID	User Story	Traces to
US-01	As a player or group of players , we want to configure a match for one to four human participants and fill any remaining seats with bot opponents, so that everyone at the table gets a seat with its own colour, start area and home path, and so that a single player can still play a full game alone.	FR-01, FR-13
US-02	As the active player , I want to draw three dice cards and pick one of them to roll, so that I can weigh the chance of rolling the maximum against the size of the move I would get.	FR-06, FR-07
US-03	As the active player , I want to choose which eligible pawn moves after the roll resolves, so that I decide between bringing a new pawn out of the yard and advancing one already on the track.	FR-02, FR-03, FR-10
US-04	As the active player , I want to send an opponent's pawn back to its yard by landing exactly on its square, so that the board stays contested and a lead can be taken away.	FR-04
US-05a	As the active player , I want to play an Action card during my own turn, so that I can change the board beyond what the die alone allows.	FR-08
US-05b	As an opponent , I want to play a Reaction card while another player acts, so that I can defend a vulnerable pawn instead of only watching.	FR-08, FR-09
US-06	As a player , I want the game to recognise the moment my fourth pawn reaches home and show a game-over screen, so that the match ends by itself and the winner is unambiguous.	FR-05

Table 2: User Stories

To better visualize the interactions and overall system flow, these user stories were mapped to a use case diagram, as shown in Figure 2: Use Case Diagram below.

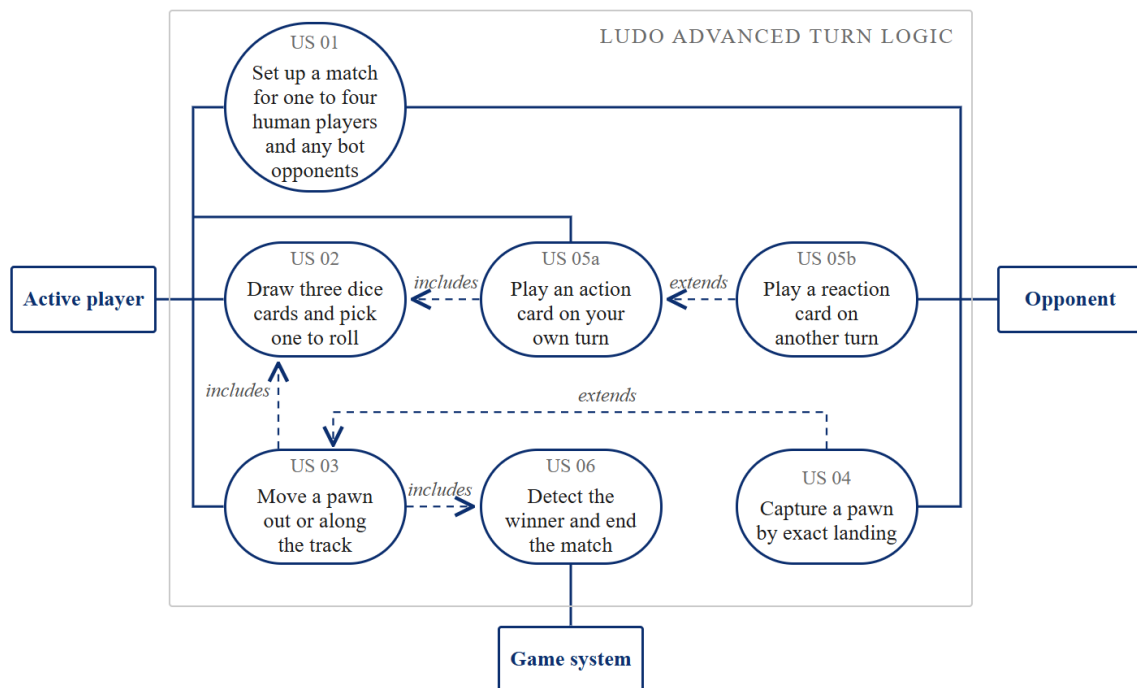


Figure 2: Use Case Diagram

3.2 Functional Requirements

Functional requirements were grouped along the four **core implementation epics**, which form the Must-Have tier:

1. Core Game Engine & Boards
2. Enhanced Dice Pool System
3. Skill Cards Mechanics
4. UI / UX & Game State.

Extended features such as online WebSocket multiplayer and LLM bot integration were scheduled as Should-Have and Won't-Have enhancements outside the MVP (See Table 10: Main Requirements and MoSCoW-Analysis).

The functional requirements were then structured into a **Product Backlog** in GitHub, which the project team could use for a Sprint-Kanban-Board:

Title	Assignees	Sprint	Labels	Status
CI: build-check workflow on every pull request #68	Ibolender	Sprint 2	4-implementation must have	Done
Core Game Engine & Board #36	BenedictGlatz a...	Sprint 2	4-implementation must have	Done
Trap Card System & Tile Trigger Logic #45	BenedictGlatz a...	Sprint 2	4-implementation could have	Done
Enhanced Dice Pool System #37	BenedictGlatz, ...	Sprint 2	4-implementation must have	Done
Skill Cards Mechanics #38	BenedictGlatz, ...	Sprint 2	4-implementation must have	Done
UI / UX & Game State #39	BenedictGlatz a...	Sprint 2	4-implementation must have	Done
i18n Setup and the German and English Locale Files #64	BenedictGlatz a...	Sprint 2	4-implementation must have	Done
Project Bootstrap: package.json, Vite, ESLint, Prettier, Vit... #63	BenedictGlatz a...	Sprint 2	4-implementation must have	Done
Create Design System #3	BenedictGlatz a...	Sprint 2	4-implementation documentation	Done

Figure 3: Backlog Epics

3.3 Non-Functional Requirements

To ensure architectural quality, high automated testability, and long-term maintainability, the team defined eight non-functional requirements as binding development constraints:

ID	Constraint	Description	Verification Method
NFR-01	Headless Game Core	<code>src/core/</code> and <code>src/state/</code> contain zero DOM or browser imports.	Automated test execution without DOM mocks.
NFR-02	300-Line Limit	No source, test, or stylesheet file exceeds 300 lines of code.	Automated lint and line-count checks.
NFR-03	Zero Hardcoded Text	All player-facing UI text is resolved dynamically via <code>i18next</code> .	Code reviews and locale coverage validation.
NFR-04	Approved Dependencies	Only vanilla JS, jQuery, and <code>i18next</code> are permitted at runtime.	Package audit and dependency review.
NFR-05	$\geq 80\%$ Coverage Floor	Lines in <code>src/core/</code> and <code>src/state/</code> must maintain $\geq 80\%$ coverage.	Vitest v8 coverage report (99.19% achieved).
NFR-06	Static Web App	Application runs entirely client-side without a mandatory server.	Static production build inspection (<code>dist/</code>).
NFR-07	Continuous Logging	Mandatory per-commit documentation and prompt tracking.	Git log and AI prompt JSON audit.
NFR-08	Transparent Refusals	UI displays clear reasons whenever an action is rejected.	State intent refusal unit and E2E tests.

Table 3: Non-Functional Requirements

4 Project Management and Team Organization

4.1 Team Structure and Roles

The project was executed by a collaborative team of three university students operating within agile roles:

Name	Primary Project Role	Core Responsibilities
Fabian Gemming	Product Owner (PO)	Requirement validation, stakeholder communication, agile assistance, design system.
Lars Bolender	Scrum Developer	Headless core rule algorithms, mathematical modeling, unit testing, State machine architecture, jQuery UI rendering, i18n localization, E2E test suites, build tooling.
Benedict Glatz	Scrum Developer	Creation of the Git-framework, game design rules, dice and skill mechanics, asset design, backlog prioritization, documentation, presentation.

Table 4: Team Structure and Roles

Fabian Gemming acted as Product Owner, holding final authority over game mechanics, balancing guidelines, acceptance criteria, and backlog prioritization. Lars Bolender and Benedict Glatz performed the development duties across all architecture layers.

Rather than dividing work by technical layers, both developers worked across the entire stack and split tasks during sprint planning. The team decided against having a dedicated Scrum Master to avoid overhead in a three-person team. Instead, the developers managed the process together, running most tasks in parallel to meet the tight project timeframe.

4.2 Development Process Model

An adapted **Scrum** process model was selected over traditional sequential approaches such as Waterfall. Game development inherently involves iterative feedback loops, especially when balancing multi-faceted card interactions and dice distributions.

The agile framework allowed the team to implement playable slices of functionality, conduct immediate playtests, and adjust rule specifications.

Furthermore, the team incorporated continuous **quality practices** into every **sprint**. Every feature was authored alongside automated unit tests, while documentation notes and AI prompt records were maintained on a per-commit basis.

4.3 Schedule and Timeline

The overall development timeline spanned eight calendar weeks from July 23, 2026, to September 17, 2026, divided into four distinct sprint iterations:

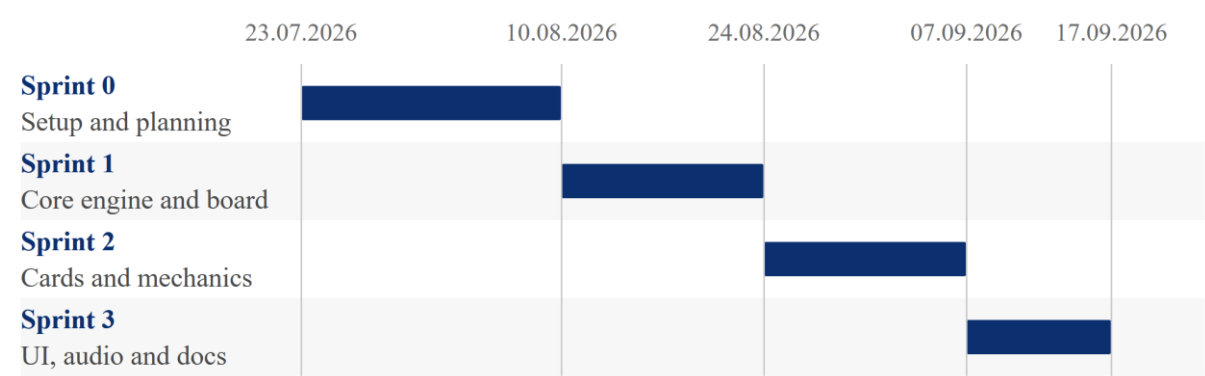


Figure 4: Schedule and Timeline

- **Sprint 0** (2.5 weeks) focused on repository bootstrapping, technology evaluation via utility value analysis, feasibility studies, and initial game rule definitions.
- **Sprint 1** (2 weeks) delivered the foundation of the headless core engine, implementing board topology, relative coordinate movement, capture rules, and the initial Vitest infrastructure.
- **Sprint 2** (2 weeks) introduced the advanced mechanics, completing the dice pool lifecycle, Action and Reaction skill cards, and the synchronous reaction window state machine.
- **Sprint 3** (1.5 weeks) concluded the project with UI polish, bilingual translation verification, Playwright E2E test runs, and final report compilation. A strict feature freeze on September 11, 2026 protected the remaining four days for final review and presentation preparation.

4.4 Collaboration and Communication

Team coordination relied on clear communication and automated version control workflows. Code was managed on GitHub using a three-tier branch hierarchy:

main held protected production releases, *dev* served as the central integration branch, and isolated *feature/issue-slug* branches housed ongoing work. Merging into *dev* strictly required a pull request and at least one peer approval.

Daily asynchronous updates and design discussions took place on Microsoft **Teams**, supplemented by **weekly synchronous sprint reviews**. Major architectural choices and rule clarifications were formally documented in *project-journal.md*, capturing the context, decisions, and rejected alternatives for full transparency.

4.5 Risk Management

Proactive risk management played a critical role in maintaining project speed and preventing scope creep throughout the semester:

Identified Risk	Probability	Impact	Active Mitigation Strategy Applied
Engine Complexity (3D Scope Creep)	High	High	Replaced Unity 3D with a lightweight 2D web build during Sprint 0 planning.
State Drift and UI Coupling	Medium	High	Enforced strict three-layer architecture isolating pure rules from view code.
Late Documentation Overhead	Medium	Medium	Mandated per-commit documentation updates and living chapter notes.
AI Tool Disruption	Low	Medium	Standardized conventions and fallback rules in <i>GEMINI.md</i> and <i>CLAUDE.md</i> .

Table 5: Risks and Mitigation Strategies

A detailed risk assessment can be found in #TODO.

4.6 Progress Tracking

Progress was continuously tracked using **GitHub** Projects v2 across three dedicated perspectives. The Kanban Board tracked task lifecycle states across *Todo*, *In Progress*, and *Done* columns, ensuring visual transparency of active work items (See Figure 12: Kanban-Board Overview).

The Backlog Table provided detailed views sorted by **MoSCoW** priority, milestone, and assigned story points. Finally, the Roadmap Gantt View mapped issue schedules against sprint boundaries, enabling early detection of critical path bottlenecks (See Figure 13: Gantt-Diagram Overview).

5 System Analysis and Design

5.1 Domain Analysis

The system analysis phase translated the physical board game into digital software domain entities. At the highest level, a *GameMatch* entity encapsulates overall game session parameters,

including the number of participating players, the currently active player index, total elapsed turns, and the current phase within the turn state machine:

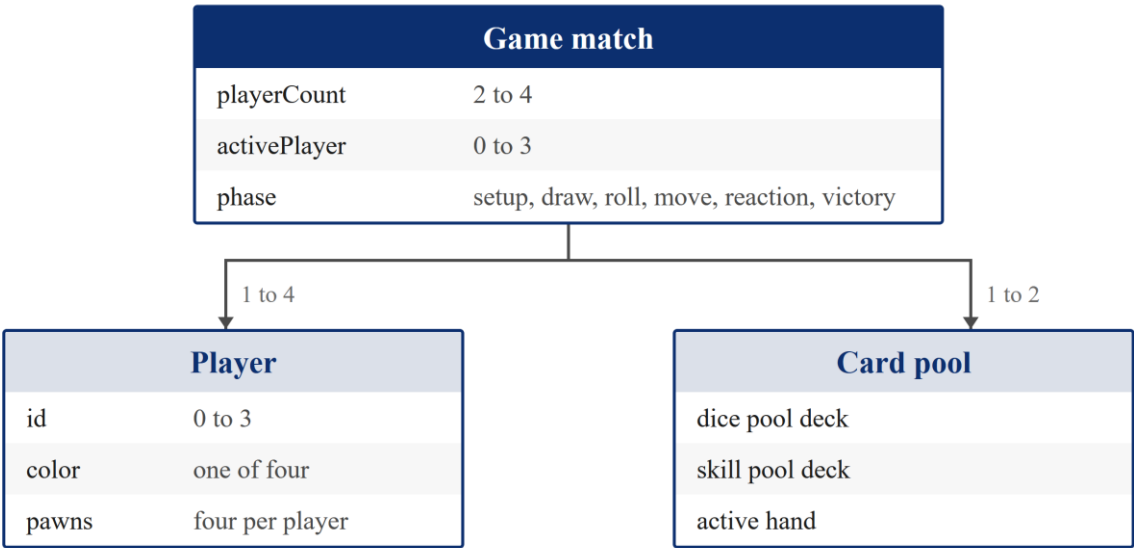


Figure 5: Domain Model

5.2 System Overview

The system is engineered as an interactive single-page application executing entirely within the client browser. The visual interface is partitioned into four primary functional zones that provide clear context throughout every match phase.

The center of the viewport is dominated by the Game Board, an interactive grid displaying the 52-tile track circuit, four distinct colored player yards, and four private home stretches. Surrounding the board, the Dice Hand Zone renders the three drawn dice cards during the active player's turn, allowing one-click selection and animated roll execution.

The Skill Card Dock provides access to available Action and Reaction cards held in hand, while the Status and Log Banner at the top of the screen displays real-time turn guidance, active player highlights, and transparent refusal messages whenever an invalid move is attempted.

5.3 Design Decisions

5.3.1 Relative Position Coordinate Topology (r)

Each player controls exactly four *Pawn* entities. Crucially, pawn positions are not stored as global coordinates on a two-dimensional grid, but as a single relative coordinate integer *r* between 0 and 40. It is calculated strictly from the owner's perspective. In this topology, *r* = 0

denotes the player's private starting yard, 1 to 40 maps onto the shared track and 41 to 44 represents the private four-step home column.

Absolute board tiles are resolved on-demand through a single pure modular formula:

$$\text{absoluteSquare}(\text{player}, r) = (\text{entrySquare}(\text{player}) + r - 1) \pmod{40}$$

Formula 1: Relative Position

5.3.2 Risk-Reward Dice Pool Dynamics (D2 to D20)

The introduction of seven polyhedral dice types fundamentally alters board game decision making. Under the classic rule that pawns require the maximum die face to exit the yard, smaller dice offer high exit reliability (such as a 50% chance on a D2) but low forward movement. On the other hand, larger dice (such as D12 or D20) provide traversal speed across the track but reduce the probability of leaving the yard.

5.3.3 Synchronous Reaction Window Architecture

To allow defensive player interaction during opponent turns without introducing multithreaded race conditions or asynchronous state locks, the system uses a synchronous Reaction Window model. Whenever an offensive Action or capture event is initiated, the engine pauses the primary action and opens a sequential reaction opportunity for affected opponents. Only after all pending reactions are resolved or declined does the primary move finalize. #TODO:Is there a chain for effects?

5.4 UI/UX Design

The visual design is structured around readability and scannability of the current board state, consistent color coding and direct interaction feedback. Because of the dice and skill cards alongside the classic board, the layout is organized to present game information and player actions within defined screen areas:

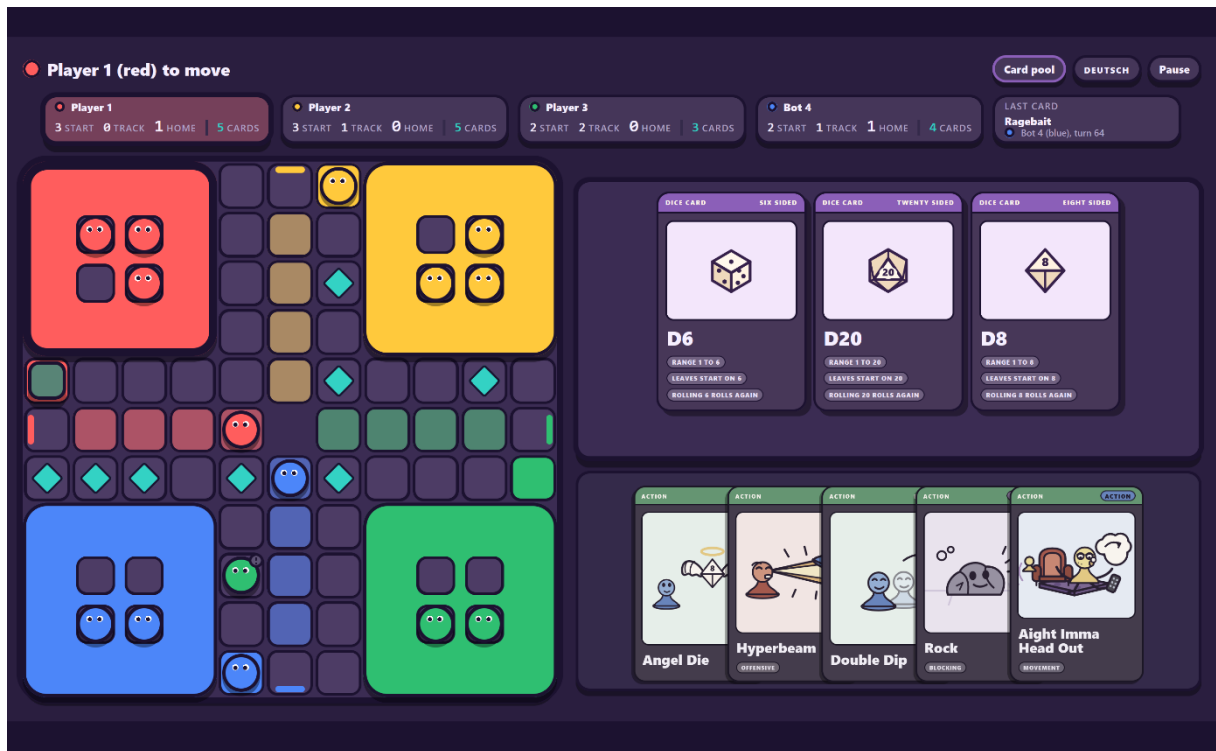


Figure 6: UI / UX in a sample Game State

The viewport is organized into distinct functional zones guided by the **Gestalt principles** of proximity and common region, which group related interface elements together visually. The primary area contains the board grid, including the common track, player yards, and home columns. An upper status panel displays turn ownership, pawn distribution counts, match logs, and general controls. The remaining interface sections include the three drawn dice cards and the player's active skill cards.

In order to keep the UI simple, the skill cards only display their detailed effect when **hovering** over them. Because human visual perception relies on pattern recognition rather than reading dense text, the card artwork allows experienced players to identify card actions faster.

Player identification applies the Gestalt law of similarity through a four-color scheme consisting of Red, Blue, Green, and Yellow. Each color maps consistently to the respective player's pawns, starting areas, home columns, card placements, and status indicators, allowing users to recognize related components across the board at a glance.

User interactions are supported through visual previews and direct error handling. Placing the cursor over an eligible pawn calculates and marks the target tile on the board, demonstrating the Gestalt law of closure by visually linking the pawn's current position to its destination.

When an invalid move is attempted, such as clicking an obstructed pawn or activating a card during an incompatible turn phase, the interface provides **immediate feedback** explaining the rule constraint.

5.5 Prototype Evolution



Figure 7: Prototype Evolution Overview

The system evolved through three structured prototyping iterations. **Phase 1** established the mathematical foundation through spreadsheet simulations of dice probabilities and card effect interactions.

Phase 2 realized the headless core engine in pure JavaScript, validating all movement, capture, and turn-sequencing rules against automated unit tests before writing any frontend code.

Finally, **Phase 3** integrated the interactive jQuery board view, CSS grid styling, and bilingual i18next translations, validating complete end-to-end player journeys in real browser environments.

6 Software Architecture

6.1 Architectural Overview

Ludo Advanced is architected around a decoupled **3-Layer Architecture** designed to isolate game logic completely from user interface rendering and browser-specific APIs. This architectural separation guarantees that business rules remain highly testable, and independent of presentation frameworks:

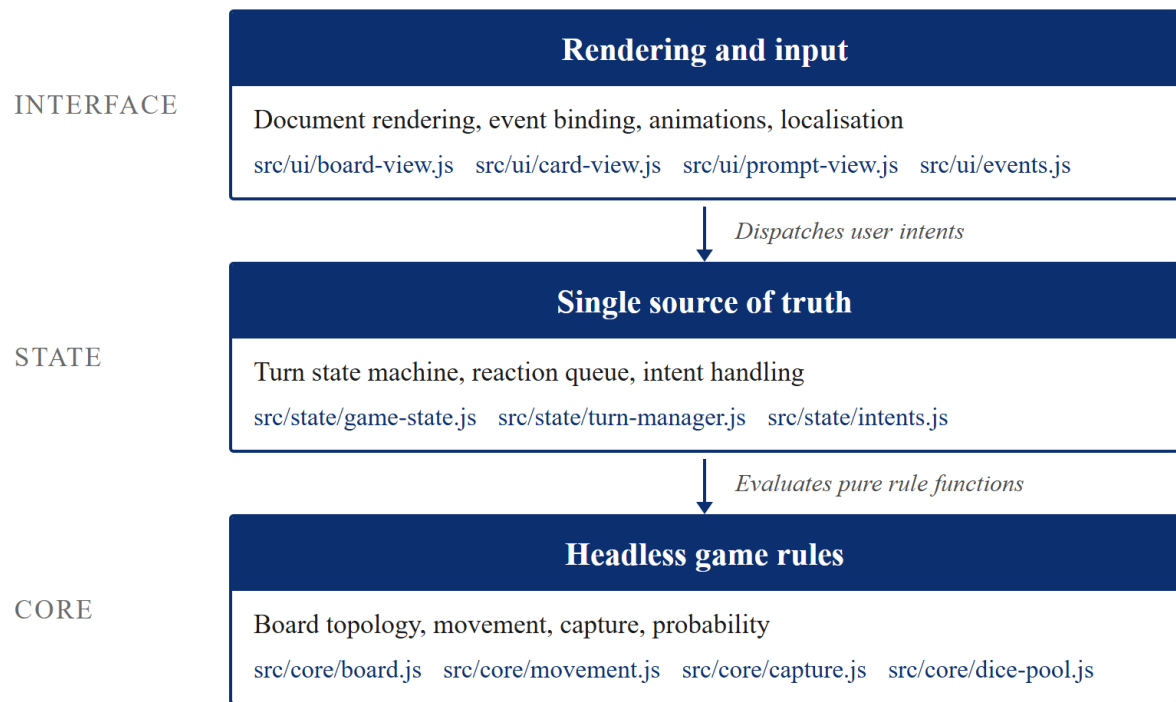


Figure 8: Architecture Layers

The Core Layer (*src/core/*) encapsulates all mathematical and algorithmic rules of the game. It calculates relative coordinates, determines valid pawn moves, evaluates captures, and manages card deck randomness without touching the DOM or window object.

The State Layer (*src/state/*) serves as the single source of truth for the match lifecycle, managing state transitions, phase sequencing, and reaction queues.

The UI Layer (*src/ui/*) reads the published state, translates state data into interactive jQuery DOM elements, binds user events, and applies CSS styling.

6.2 Architectural Decisions

To ensure maintainability and testability, three structural principles govern the codebase:

First, the system enforces **Strict Upward Isolation**. Modules in the Core Layer are forbidden from importing any code from the State or UI Layer. Similarly, the State Layer never imports from the User Interface. This guarantees that core logic can be executed and unit-tested in pure Node.js environments without browser simulation or DOM mocks.

Second, state changes are managed via an **Intent-Driven Mutation Pattern** to decouple the UI from state handling. The UI Layer cannot modify state directly; instead, user interactions, such as selecting a card or a pawn, dispatch structured intent objects to *src/state/intents.js*. The

state manager evaluates each intent against the core rule validation logic. Valid intents produce a new immutable state object that is passed down to update the view, whereas invalid intents append an error code and refusal message to the state to trigger localized feedback.

Third, all card mechanics are implemented as **Pure Functional Transformers**. Dice and skill card effects receive the current immutable game state alongside target parameters, computing and returning an updated state object without side effects.

6.3 Component Diagram

The application codebase is partitioned into distinct functional subsystems that map directly to project requirements:

Architecture Layer	Key Modules	Primary Responsibilities
Composition Root	<i>src/main.js</i>	Initializes i18next localization, instantiates state store, mounts UI listeners, and boots game loop.
UI Presentation	<i>src/ui/board-view.js</i> , <i>src/ui/card-view.js</i> , <i>src/ui/prompt-view.js</i>	Renders board tiles, cards, and modal dialogs; captures DOM click events; updates CSS classes.
State Machine	<i>src/state/game-state.js</i> , <i>src/state/turn-manager.js</i> , <i>src/state/reaction-window.js</i>	Holds immutable state store, sequences 8-step turn lifecycle, coordinates opponent reaction windows.
Core Game Rules	<i>src/core/board.js</i> , <i>src/core/movement.js</i> , <i>src/core/capture.js</i> , <i>src/core/dice-pool.js</i>	Computes relative coordinate math, evaluates legal pawn moves, executes captures, manages dice decks.
Localization	<i>src/i18n/index.js</i> , <i>src/i18n/locales/*.json</i>	Asynchronously resolves all UI labels and refusal explanations into German and English.

Figure 9: Component Diagram

6.4 Frontend Design

The frontend architecture prioritizes fast rendering and minimal overhead using jQuery 4.0 and Vanilla CSS. To avoid binding hundreds of individual click handlers to individual board squares and cards, the UI employs container-level event delegation.

When user interactions occur, delegated listeners extract data attributes (such as ``data-player`` or ``data-pawn-index``) and construct intent payloads. Rendering updates are state-driven: view

components subscribe to state broadcasts and selectively re-render only the affected subtrees, such as updating pawn token positions or toggling card availability buttons.

6.5 Backend Design

#TODO

Because the project Minimum Viable Product focuses on local hot-seat gameplay on a single shared device, the application was deliberately built as a ****serverless static web application****. Eliminating a dedicated backend server removed deployment complexity, hosting costs, and network latency risks.

However, because all player actions pass through structured intent dispatchers in `src/state/intents.js`, the architecture is naturally prepared for future networked multiplayer extensions (Epic #42). Supporting remote multiplayer would simply require swapping the local intent dispatcher for a WebSocket network transport adapter that streams serialized state trees between clients.

6.6 Runtime / Deployment View

The deployment and runtime architecture reflects the project's design as a standalone, serverless **static web application**. The system operates entirely within client environments, requiring no persistent application servers, runtime databases, or execution hosts.

The deployment pipeline is fully automated via **GitHub Actions** continuous deployment workflows. When changes are integrated and pushed to the protected branches, the pipeline executes the automated verification suite, compiling the source files with Vite into an optimized, static distribution bundle (*dist/*).

This production build is published directly to **GitHub Pages**, where the bundled HTML, JavaScript, CSS, and static vector assets are served statically over HTTPS via a **Content Delivery Network** (CDN).

At runtime, the execution lifecycle occurs entirely inside the user's web browser thread. When a user accesses the URL, the browser retrieves the static bundle and bootstraps the ES Modules. The application initializes the core rule engine, binds the localized strings via i18next into memory, sets up the reactive game state store, and attaches delegated event listeners to the DOM through jQuery.

All state transitions, dice probability simulations, and turn validations execute **locally in browser memory** without intermediate client-server roundtrips. Match sessions remain volatile

within the active browser tab lifecycle, avoiding local cache accumulation or external API dependencies.

7 Data Design

7.1 Data Model

7.2 Entity Relationship Diagram

7.3 Data Sources and Data Flow

8 Implementation

8.1 Development Environment

8.2 Project Structure

8.3 Key Features

8.4 Selected Implementation Details

8.5 Use of External Libraries and AI Tools

9 Testing and Quality Assurance

9.1 Testing Strategy

The project uses a multi-tier quality assurance strategy that separates pure mathematical rule validation from visual browser rendering:

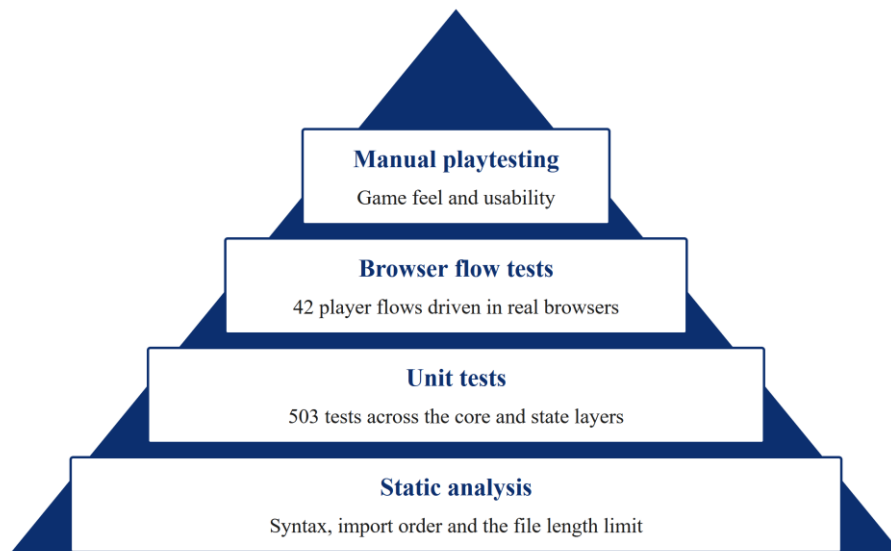


Figure 10: Testing Pyramid

#TODO:Nach Codefreeze Zahlen anpassen

At the foundation, static code analysis tools (**ESLint** and **Prettier**) enforce syntax correctness, module import boundaries, and the 300-line file limit.

The unit testing tier covers all pure game rules in *src/core/* and state transitions in *src/state/* using Vitest. Because these modules contain zero DOM dependencies, the entire suite executes in under two seconds without browser simulation.

The integration and UI tiers rely on Playwright for automated end-to-end testing, simulating complete player matches across Chromium, Firefox, and WebKit browsers. Finally, manual playtesting sessions provided qualitative feedback on game feel, rule comprehension, and visual pace.

9.2 Test Cases

Automated test suites were designed to cover all 45 functional requirements and edge cases defined in the game design documentation:

Test Area	Scope and Modules Covered	Suite Count	Test Count	Pass Rate
-----------	---------------------------	-------------	------------	-----------

Board & Movement	<i>board.js,</i> <i>movement.js,</i> <i>pawns.js,</i> <i>path.js</i>	8 files	124 tests	100%
Dice Mechanics	<i>dice-pool.js,</i> <i>roll.js,</i> <i>dice-source.js</i>	6 files	86 tests	100%
Skill Cards	<i>skill-pool.js,</i> <i>skill-turn.js,</i> <i>reaction-window.js</i>	10 files	158 tests	100%
State Machine	<i>game-state.js,</i> <i>turn-manager.js,</i> <i>intents.js</i>	10 files	135 tests	100%
End-to-End (E2E)	Complete browser match flows in Chromium, Firefox, WebKit	9 files	42 tests	100%
Total Auto-mated	Full Project Verification	43 files	545 tests	100%

Figure 11: Testing, Count and Pass Rate

9.3 Test Results

Code coverage was evaluated using the v8 engine in Vitest, measured on 2026-08-31:

Directory	Target	Line Coverage	Branch Coverage	Function Coverage
<i>src/core/</i> (27 files)	≥ 80.0%	99.59% (483 / 485)	96.42%	100.0% (182 / 182)
<i>src/state/</i> (10 files)	≥ 80.0%	98.48% (259 / 263)	95.16%	100.0% (84 / 84)
Total Combined	≥ 80.0%	99.19% (742 / 748)**	96.00%	100.0% (266 / 266)

Table 6: Coverage Results

The combined line coverage of 99.19% significantly exceeds the project requirement floor of 80% (NFR-05). Function coverage achieved 100% across all 266 declared functions, demonstrating that virtually every execution path in the core rules and state engine is verified by automated regression tests.

9.4 Defects and Corrections

Throughout the testing phase, automated suites helped identify and resolve several non-obvious defects.

Early in development, test coverage tables appeared unexpectedly empty; investigation revealed that Vitest's default text reporter omitted files with 100% coverage, a quirk resolved by configuring the `json-summary` reporter in `vitest.config.js`.

During skill card integration, simultaneous reaction triggers from multiple players caused race conditions, prompting a redesign into a deterministic, sequential reaction queue in `src/state/reaction-window.js`.

Additionally, an edge case where dice rolls exceeded the remaining distance to the goal caused pawns to freeze; this was corrected in `src/core/movement.js` by adding an exact-bounce calculation that moves pawns backwards on surplus values.

9.5 Quality Improvements

Quality enforcement was embedded directly into the daily workflow. Refusal transparency (NFR-08) was enforced by mapping every rejected user intent to an explicit, localized refusal code. Furthermore, running npm test in a pure Node.js context without DOM shims guarantees that core game rules remain permanently isolated from presentation logic.

10 Reflection and Evaluation

10.1 Goal Achievement

The project successfully delivered all planned Must-Have epics and exceeded its technical and organizational benchmarks within the eight-week development window:

Objective	Target Baseline	Final Outcome	Evaluation
Playable Core MVP	Epics 36-39 complete	100% Implemented & Validated	Completed
Code Modularity	Max 300 lines per file	Longest file is 295 lines	Achieved
Test Coverage	$\geq 80.0\%$ line coverage	99.19% line coverage	Exceeded
Bilingual Localization	Full DE / EN translation	100% text resolved via i18next	Achieved
Headless Isolation	Zero DOM imports in core/	100% headless rules	Achieved

Table 7: Objectives and Evaluation

All core game mechanics, including the 52-tile board topology, relative coordinate tracking, polyhedral dice pool selection, Action and Reaction skill cards, and dynamic UI rendering, were verified and deployed on schedule.

10.2 Technical Lessons Learned

10.2.1 Headless Architecture enables Rapid Test Automation

The decision to strictly decouple game rules and state management from the DOM proved to be the most impactful architectural choice of the project. Because business logic operates as pure functions over plain JavaScript objects, 503 unit tests execute in under two seconds without requiring mocking libraries or DOM shims. This fast feedback loop encouraged continuous test authoring and refactoring throughout development. For future interactive web applications, **isolating core domain rules from UI** presentation should always be prioritized from day one.

10.2.2 Normalized Relative Coordinate Spaces Reduce Complexity

Modeling pawn positions along a **personal, relative coordinate integer** radically simplified game logic compared to traditional two-dimensional grids. By calculating movement, yard exit eligibility, capture conditions, and victory states in relative terms, a single set of pure functions serves all four players uniformly. Board offsets are applied only at the final rendering stage, eliminating redundant conditional branches across the codebase.

10.3 Project Management Lessons Learned

While pivoting from a 3D Unity implementation to a 2D web application was vital to reduce technical complexity, it did not eliminate the severe schedule pressure. Ludo Advanced demanded the largest scope of any academic project the team had undertaken, ultimately requiring an intense **crunch phase** to reach completion. Delivering the project required balancing a massive codebase alongside extensive formal project management deliverables, custom card assets, and playtesting loops.

To overcome this, the team had to establish a new level of **parallel execution** across all members. However, this high degree of parallelization was introduced too late in the lifecycle. During the initial phases, the team over-invested in serialized documentation to build a perfectly aligned, shared mental model of every requirement before starting implementation. While comprehensive specification prevented misalignment later, it yielded **diminishing returns** by consuming critical development time.

In a compressed eight-week timeframe, striving for universal consensus across every minute detail is inefficient. An agile framework permits overlapping initialization, planning, and architectural definition with early implementation for already defined modules. Running execution and formal requirement detailing concurrently from Sprint 0 would have relieved downstream bottlenecks.

10.4 Teamwork Reflection

#TODO

Collaboration within the three-person team was characterized by clear role ownership and high transparency. Fabian Gemming's dedicated focus as Product Owner on rule design and backlog clarity allowed Lars Bolender and Benedict Glatz to concentrate on software architecture, headless algorithms, and UI integration. Mandatory pull request reviews ensured shared code ownership and prevented regressions.

11 Conclusion

11.1 Summary

The Ludo Advanced project successfully modernized the traditional roll-and-move board game into a strategical, highly engaging digital web application. Over an eight-week development cycle, the team engineered a single-page web application that balances chance and strategy through dice selection (D2 to D20) and Action/Reaction skill cards.

From an engineering perspective, the project demonstrated the practical value of a **strictly decoupled 3-layer architecture** (core, state and ui), allowing game rules to remain completely independent of the DOM. This architectural purity enabled extensive automated testing, yielding an exceptional 99.19% line coverage across the core rule engine and state machine. The project concluded on schedule, fulfilling all Must-Have epics, adhering strictly to modular 300-line file constraints, providing complete bilingual localization, and maintaining full academic traceability through continuous per-commit documentation.

11.2 Open Issues

At the conclusion of the final sprint, the **core Minimum Viable Product operates** with zero blocking functional defects. All rule calculations, movement validations, capture mechanics, reaction window timings, and victory conditions function reliably across supported desktop browsers.

Minor non-blocking items identified for future polish include the integration of synthesized **audio** cues for dice rolling and pawn capture events, as well as **layout optimizations**.

11.3 Future Work

Looking forward, the clean architectural boundaries established in the project provide a robust foundation for future extensions. Solo gameplay could be enhanced with autonomous **AI bots** powered by heuristic state evaluation or large language model APIs. Finally, adding dynamic **audio effects** for dice rolling and pawn captures would further heighten visual immersion.

12 Appendix

12.1 Tables

Dimension	Target Baseline	Achieved Result	Evaluation
Playable Scope (MVP)	4 Core Epics (#36 to #39)	Fully implemented local hot-seat MVP	Completed
Rule Test Coverage	≥ 80% line coverage in core/ & state/	99.19% line coverage (742 / 748 lines)	Exceeded
Automated Tests	Full rule and UI regression suites	503 unit tests + 42 E2E browser tests	Exceeded
Headless Architecture	Zero DOM imports in core/ (NFR-01)	Pure headless core with 0 DOM dependencies	Achieved
Code Modularity	Max 300 lines per file (NFR-02)	100% compliant across source and tests	Achieved

Table 8: Key Outcomes

	Criteria	Description
S	Specific	• Player count: 1 to 4. • Dice Card Pool: draw 3 cards from a pool ranging from D2 to D20, pick 1, roll it, shuffle all 3 back. • Skill Card Pool: Action cards on the player's own turn, Reaction cards in response to another player. • Win path: all pawns from the start area to home. A build that renders a board but never reaches a victory does not qualify as done.
M	Measurable	• Five independent checks: a full game completes with 1, 2, 3 and 4 players • All four must have epics are closed on the Kanban-board • The coverage command reports at least 80 % line coverage for the rules and state layers. • The German and English locale files resolve every visible string, with no hardcoded text remaining. • No source file exceeds 300 lines.
A	Achievable	• The scope was cut to a 2D web build so the team could stay in JavaScript, which is already known, rather than learning C# for a 3D engine. • The utility value analysis scored that option 4.20 of 5.00 against 2.30 for a full 3D build, weighted on team competence and available time.
R	Relevant	Through the specifications (S) and requirements (FR, NFR), the goals of the module “Projektpraktikum” are satisfied: • Practice technical skills by collaborating in a team to design and develop a complete (software) product • Independently manage and structure the end-to-end software development lifecycle • Enhance communicative and social competence through active teamwork and group collaboration
T	Time-bound	• The end-date is set to 17.09.2026, the end of the last planned sprint and marks the date for delivering the documentation, as well as the presentation.

Table 9: SMART-analysis

ID	Requirement Description	Epic Category	MoSCoW
FR-01	Support match configuration for 1 - 4 players.	Core Engine	Must Have
FR-02	Track relative pawn positions on a 40-tile circuit.	Core Engine	Must Have
FR-03	Enforce yard exit rule requiring the maximum die face.	Core Engine	Must Have
FR-04	Execute exact-landing pawn capture and yard return.	Core Engine	Must Have
FR-05	Detect victory when 4 pawns reach the home goal.	Core Engine	Must Have
FR-06	Provide Dice Pool deck containing D2, D4, D6, D8, D10, D12, and D20.	Dice Pool	Must Have
FR-07	Implement draw-3, select-1, and shuffle-back turn cycle.	Dice Pool	Must Have
FR-08	Implement Skill Pool with Action and Reaction card types.	Skill Cards	Must Have
FR-09	Open dynamic Reaction windows upon opponent trigger actions.	Skill Cards	Must Have
FR-10	Provide responsive 2D board view with pawn movement highlights.	UI / UX	Must Have
FR-11	Full bilingual UI localization in German and English.	UI / UX	Must Have
FR-12	Online multiplayer lobby and network synchronization.	Networking	Should Have
FR-13	Autonomous AI bot opponents powered by heuristic or LLM models.	AI Opponents	Won't Have

Table 10: Main Requirements and MoSCoW-Analysis

12.2 Figures

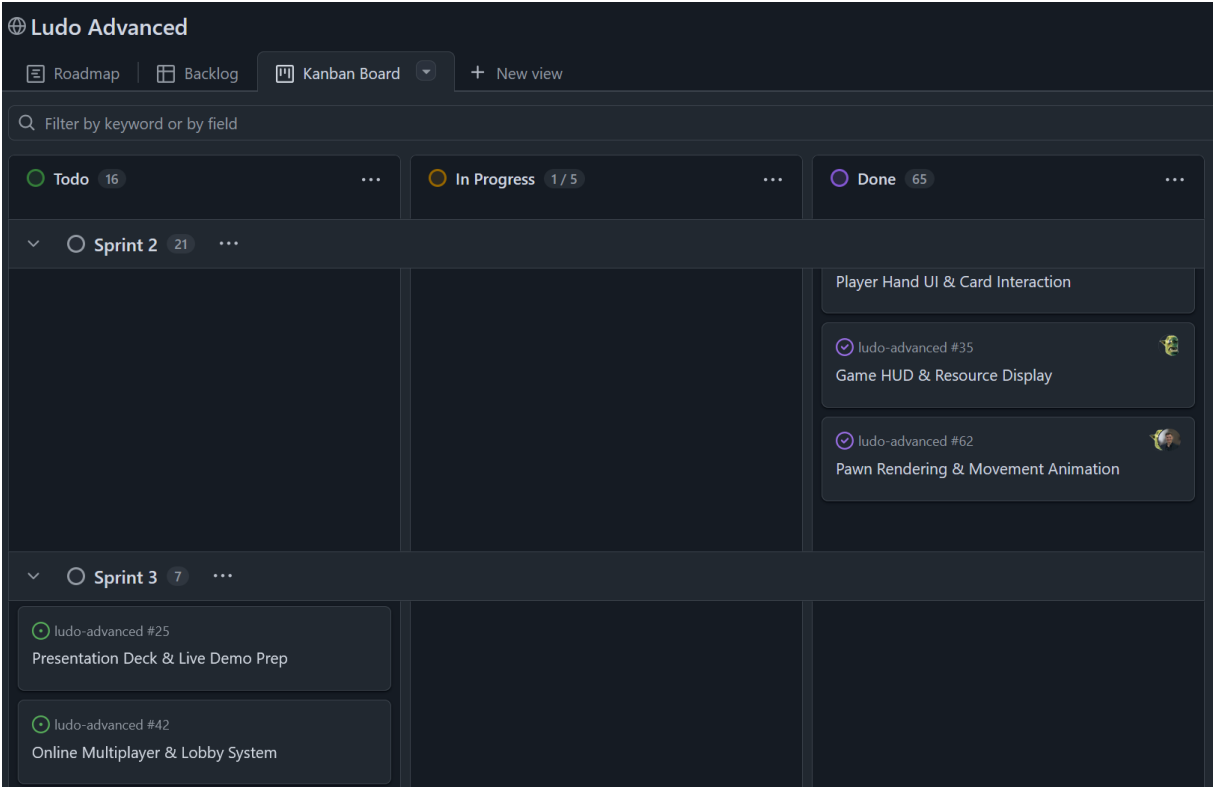


Figure 12: Kanban-Board Overview

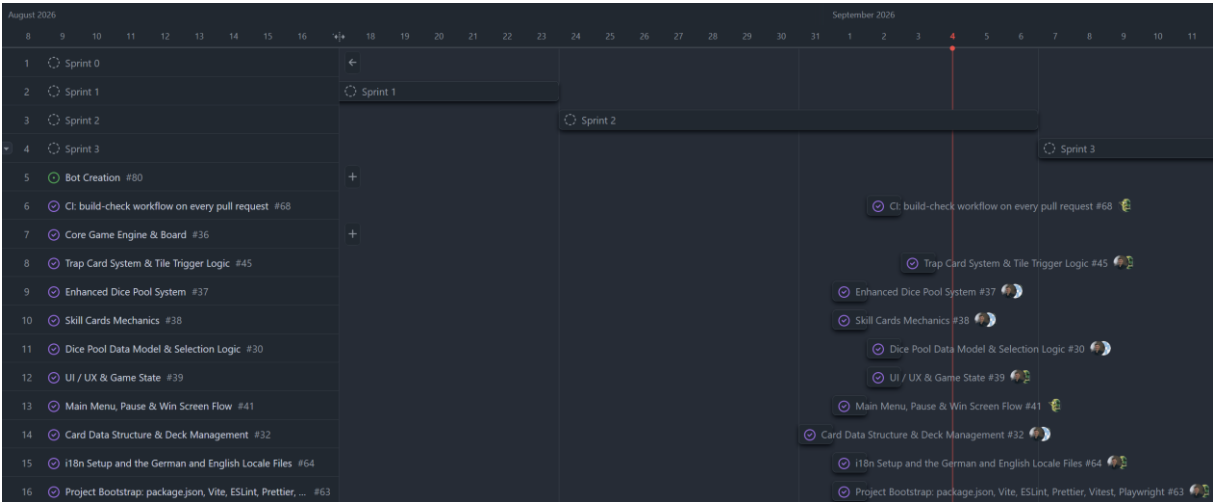


Figure 13: Gantt-Diagram Overview

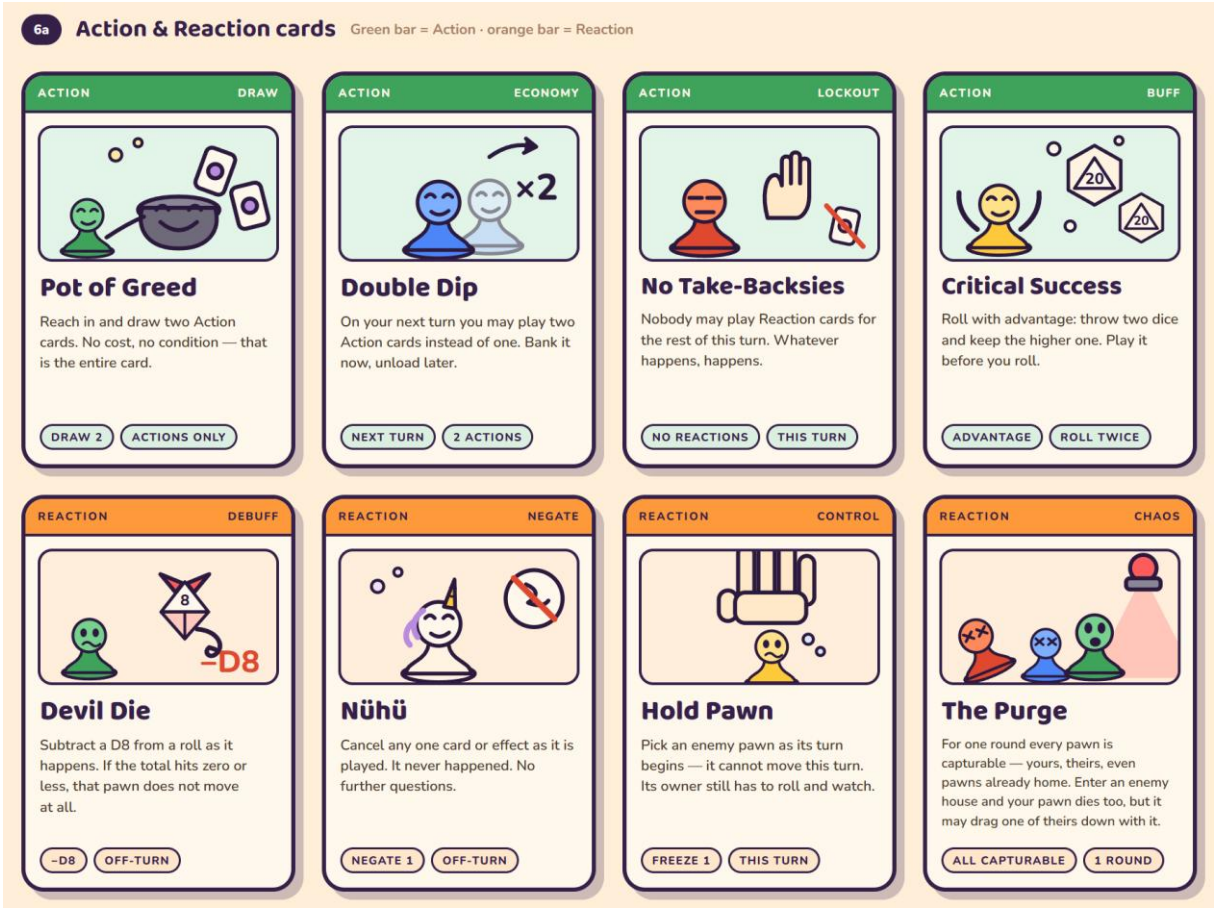


Figure 14: Action & Reaction Card Mockup

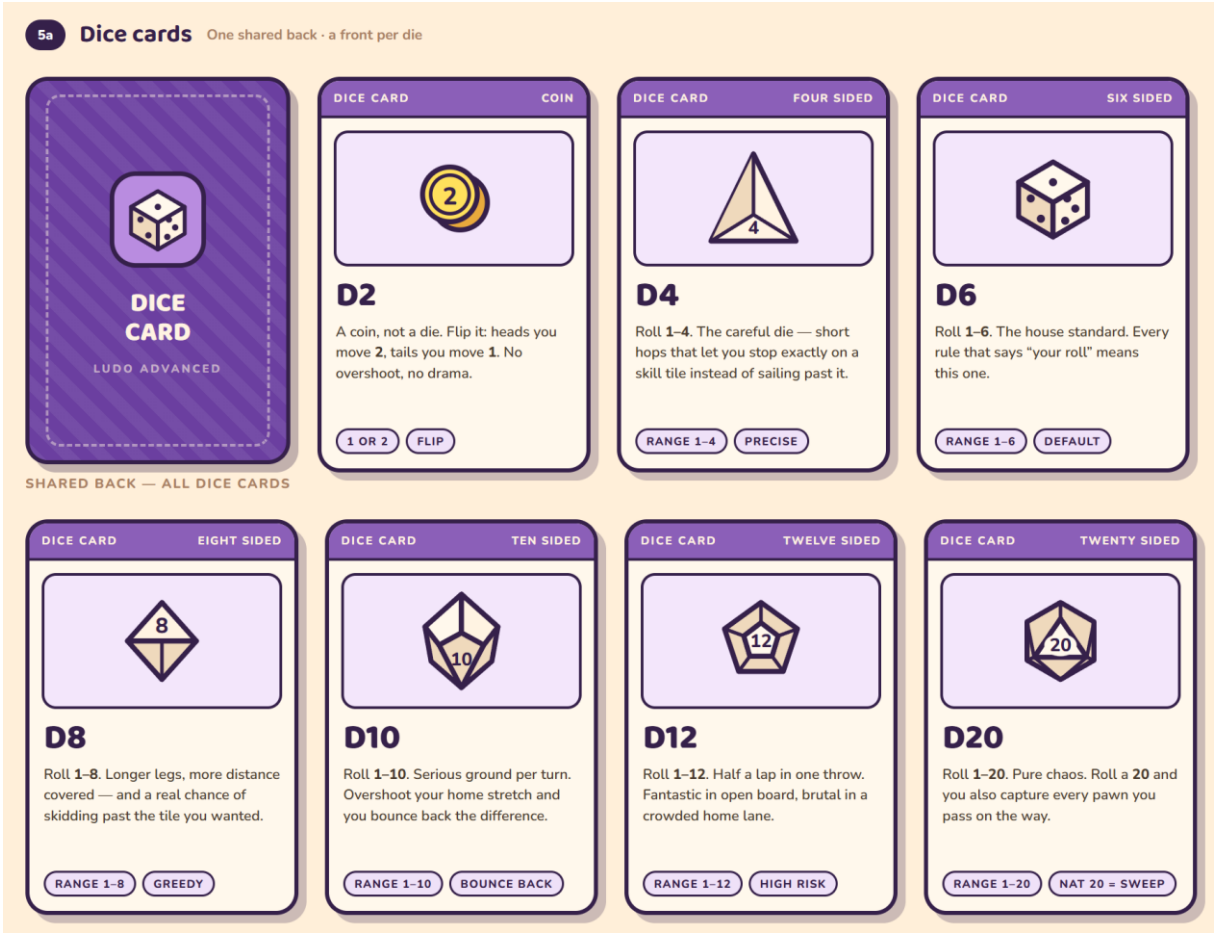


Figure 15: Dice Card Mockup

13 AI Directory

System	Prompt	Application
ChatGPT 4o	Erstelle einen Einleitungssatz für mein Kapitel „Literatur-Review“ wie „In diesem Abschnitt wird ein wissenschaftliches Fundament erarbeitet, auf welches meine Arbeit aufbaut“.	Abgeändert
ChatGPT 4o	Aufgrund der sich rapide verändernden ... ist die Prozessautomatisierung nicht nur ein Vorteil, sondern eine Notwendigkeit, insbesondere für Unternehmen, die ihre Wettbewerbsfähigkeit erhalten wollen. Setze hier etwas ein wie Geschäftswelt, Bedingungen in der IT, unternehmerischen Umstände. Für eine wissenschaftliche Arbeit	Unverändert
ChatGPT 4o	Gib mir mögliche Keywords für Google Scholar zum Thema „Einfluss eines integrierten Tools zur Abwesenheitsverwaltung auf Effizienz und Fehleranfälligkeit“	Nur Recherche
ChatGPT 4o	Wenn meine Seitenzahl knapp wird, wie würdest du den Gliederungsentwurf in das Dokument integrieren?	Nur Recherche
ChatGPT 4o	Wie ist in einer wissenschaftlichen Arbeit die Gliederung üblicherweise unterteilt?	Abgeändert
ChatGPT 4o	Kürze, ohne Inhalt zu verlieren: Häufig werden die Prozesse zur Abwesenheitsverwaltung in Unternehmen manuell und in unterschiedlichen Systemen gepflegt, was einen hohen administrativen Aufwand und eine erhöhte Fehleranfälligkeit zur Folge hat. Ein Tool, das direkt in die primär genutzte Kommunikationstechnologie integriert ist, bietet die Möglichkeit, diese Prozesse zu optimieren und zu vereinfachen.	Unverändert
ChatGPT 4o	Kürze, ohne Inhalt zu verlieren: Mit dieser Arbeit soll gezeigt werden, dass die Integration eines automatisierten Tools in eine Kommunikationsplattform wie Outlook eine signifikante Prozessverbesserung in der Abwesenheitsverwaltung ermöglicht. Die Erkenntnisse sollen Unternehmen helfen, Prozesse effizienter und fehlerfreier zu gestalten.	Unverändert
ChatGPT 4o	Kürze, ohne Inhalt zu verlieren: Zur Beantwortung der Forschungsfrage wird zunächst ein VBA-Programm implementiert, welches in Outlook integriert wird. Anschließend werden zwei Vergleichsgruppen gebildet. Gruppe A verwendet weiterhin den manuellen Prozess, wohingegen Gruppe B das implementierte Programm zur Abwesenheitserfassung nutzt. Die Zeitmessung erfolgt über standardisierte Testabläufe (Urlaub, Krankheit) und Fehler werden anhand fehlender oder inkorrekt eingetragener dokumentiert. Die Signifikanzprüfung der ersten Hypothese erfolgt mithilfe eines t-Tests.	Unverändert
ChatGPT 4o	Ein VBA-Tool wird in Outlook integriert. Zwei Vergleichsgruppen testen typische Anwendungsfälle (z. B. Krankmeldung). Gruppe A nutzt den bisherigen manuellen Prozess, Gruppe B das neue Tool. Erfasst werden Bearbeitungszeit und Fehlerhäufigkeit. Die Auswertung erfolgt quantitativ; ein t-Test prüft die erste Hypothese auf Signifikanz.	Abgeändert

Mache diesen Text fließender